

Sliding Window Templates

Three variants — pick by what the problem asks for.

i = left boundary (inclusive), j = right boundary (loop variable, inclusive).

Window = $[i, j]$, size = $j - i + 1$.

The Three Templates

```
// FIXED window of size k — shrink exactly once when full
// MENTAL MODEL: a constant-width window slides one step at a time; add the new cell, drop the old one.
// WHEN: "substring/substring of size k"
int i = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    if (j - i + 1 == k) {
        // 2. record (window is exactly size k)
        // 3. remove nums[i] from state
        i++;
    }
}

// MAX variable window — find LONGEST valid window
// MENTAL MODEL: grow greedily; shrink only enough to restore validity, then the window is the best ending here.
// WHEN: "longest" + "at most k ..."
int i = 0, result = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window exceeds constraint */) { // shrink WHILE window exceeds constraint
        // remove nums[i] from state
        i++;
    }
    result = Math.max(result, j - i + 1); // record after window is valid
}

// MIN variable window — find SHORTEST valid window
// MENTAL MODEL: grow until valid, then shrink aggressively while still valid to find the tightest fit.
// WHEN: "shortest/minimum" + "sum >= target" / "contains all of t"
int i = 0, result = Integer.MAX_VALUE;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window VALID */) { // record then shrink WHILE window still valid
        result = Math.min(result, j - i + 1); // record while valid
        // remove nums[i] from state
        i++; // then try to shrink
    }
}
```

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
6 4 3	Maximum Average Subarray I	Find the contiguous subarray of length <code>k</code> with the maximum average value.	Max average of fixed length = max sum; slide a width- <code>k</code> window and track the best running sum.	$O(n)$	$O(1)$	Standard
5 6 7	Permutation in String	Return true if any permutation of <code>s1</code> appears as a substring of <code>s2</code> .	A permutation is just a fixed-length window whose letter counts match <code>s1</code> , so slide and check the counts.	$O(n)$	$O(1)$	Standard
2 1 9	Contains Duplicate II	Return true if any two equal values are at most <code>k</code> indices apart.	Keep only the last <code>k</code> values in a set; a failed insert means a duplicate within <code>k</code> indices.	$O(n)$	$O(k)$	Standard
2 3 9	Sliding Window Maximum	Return the maximum of every contiguous subarray of size <code>k</code> .	A smaller value with a larger one still in the window can never be the max again, so discard it.	$O(n)$	$O(k)$	Standard
1 0 0 4	Max Consecutive Ones III	Flip at most <code>k</code> zeros. Return the length of the longest subarray of 1s.	"Flip at most <code>k</code> zeros" = longest window containing at most <code>k</code> zeros; grow, and shrink only when zeros exceed <code>k</code> .	$O(n)$	$O(1)$	Standard
3	Longest Substring Without Repeating Characters	Find the length of the longest substring with all unique characters.	A repeat appears the moment you add it, so shrink from the left until that one character is unique again.	$O(n)$	$O(1)$	Standard
4 2 4	Longest Repeating Character Replacement	Replace at most <code>k</code> characters. Find the longest substring with one repeated char.	A window is valid if the non-dominant chars (size – <code>maxFreq</code>) fit within <code>k</code> replacements; otherwise shrink.	$O(n)$	$O(1)$	Standard
2 0 9	Minimum Size Subarray Sum	Find the minimum length subarray with sum $\geq$ target.	Once the window reaches target, every extra left-trim that stays $\geq$ target gives a shorter candidate.	$O(n)$	$O(1)$	Standard
7 6	Minimum Window Substring	Find the shortest substring of <code>s</code> that contains all characters of <code>t</code> .	Expand until all of <code>t</code> is covered, then shrink as far as you can while still covering it to find the tightest window.	$O(n)$	$O(1)$	Standard
4 3 8	Find All Anagrams in a String	Given strings <code>s</code> and <code>p</code> , return a list of all start indices of <code>p</code> 's anagrams in <code>s</code> . (An anagram uses same characters with same frequencies.)	An anagram is a fixed-length window whose letter counts equal <code>p</code> 's, so slide width- <code>p.length()</code> and compare counts.	$O(n)$	$O(1)$	fixed window size
1 5 9	Longest Substring with At Most 2 Distinct Characters	Return the length of the longest substring with at most 2 distinct characters.	Grow the window freely; whenever a third distinct character appears, drop from the left until only 2 remain.	$O(n)$	$O(1)$	shrink when >2 distinct
3 4 0	Longest Substring with At Most K Distinct Characters	Return the length of the longest substring with at most <code>k</code> distinct characters.	Same as #159 but the distinct cap is <code>k</code> ; shrink whenever the map holds more than <code>k</code> distinct characters.	$O(n)$	$O(k)$	Parameterized generalization of #159 – replace the hard-coded 2 with <code>k</code> .
1 3 4 3	Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold	Return the number of contiguous subarrays of size <code>k</code> whose average is greater than or equal to <code>threshold</code> .	"Average $\geq$ threshold" over fixed width <code>k</code> is just "sum $\geq$ <code>k * threshold</code> "; slide a width- <code>k</code> window and count the hits.	$O(n)$	$O(1)$	fixed-size sliding window. To avoid floating-point division, compare <code>windowSum >= k * threshold</code> .
3 0	Substring with Concatenation of All Words	Given <code>s</code> and an array <code>words</code> (all of equal length), return all start indices of substrings in <code>s</code> that are a concatenation of every word in <code>words</code> exactly once, in any order.	Every candidate substring has fixed length <code>words.length * wordLen</code> . Slide a word-aligned window: start at each offset <code>0..wordLen-1</code> and move in steps of <code>wordLen</code> , maintaining a frequency map of words seen.	$O(n * \text{wordLen})$	$O(\text{wordLen} * \text{wordLen})$	word-aligned window, step = <code>wordLen</code>
1 8 7	Repeated DNA Sequences	Return all 10-letter substrings that occur more than once in a DNA string <code>s</code> (characters <code>A</code> , <code>C</code> , <code>G</code> , <code>T</code>).	Fixed window of size 10; record each substring in a set and report it the first time a duplicate insert fails.	$O(n)$	$O(n)$	fixed window size 10
3 9 5	Longest Substring with At Least K Repeating Characters	Return the length of the longest substring of <code>s</code> such that every character in it appears at least <code>k</code> times.	"At least <code>k</code> " is not monotone for a single window, so fix the number of distinct characters allowed. For each target <code>unique</code> from 1 to 26, run a max-window that holds exactly <code>unique</code> distinct chars, and record when all of them meet the count <code>k</code> .	$O(26 * n)$	$O(1)$	fix distinct count to restore monotonicity
4 8 7	Max Consecutive Ones II	Given a binary array <code>nums</code> , return the maximum number of consecutive 1s if you may flip at most one 0.	Identical to #1004 with <code>k = 1</code> : longest window containing at most one zero. Grow, and shrink only when a second zero enters.	$O(n)$	$O(1)$	at most one zero (<code>k = 1</code>)
6 8 9	Maximum Sum of 3 Non-Overlapping Subarrays	Find three non-overlapping subarrays of length <code>k</code> with maximum total sum and return their starting indices (lexicographically smallest on ties).	Precompute every window sum of size <code>k</code> , then for each middle window pick the best left window to its left and the best right window to its right.	$O(n)$	$O(n)$	middle window scan with <code>k</code> -gaps

#	Name	Description	Intuition	Time	Space	Key Implementation
7 1 3	Subarray Product Less Than K	Return the number of contiguous subarrays whose product of all elements is strictly less than <code>k</code> .	Max-variable window: grow the window while the product stays below <code>k</code> ; every valid window ending at <code>j</code> contributes <code>j - i + 1</code> new subarrays.	$O(n)$	$O(1)$	count subarrays ending at <code>j</code>
7 2 7	Minimum Window Subsequence	Return the minimum-length substring (window) of <code>s1</code> such that <code>s2</code> is a subsequence of it. On ties, return the leftmost.	Walk forward matching <code>s2</code> as a subsequence; once fully matched at index <code>j</code> , walk backward to tighten the start, giving the smallest window ending at <code>j</code> . Restart the forward scan just past that start.	$O(n * m)$	$O(1)$	subsequence match, not contiguous
1 0 4 4	Longest Duplicate Substring	Return any longest substring that appears at least twice in <code>s</code> (overlaps allowed); return <code>""</code> if none.	Binary search the answer length: if a duplicate of length <code>L</code> exists, a duplicate of any shorter length also exists. Check each candidate length with a rolling hash over a fixed-size window.	$O(n \log n)$ average	$O(n)$	binary search the window size
1 8 3 8	Frequency of the Most Frequent Element	Given <code>nums</code> and <code>k</code> operations (each increments one element by 1), return the maximum possible frequency of any single value.	Sort the array; the cheapest target to raise a window of elements to is its maximum (the rightmost in a sorted window). A window <code>[i, j]</code> is achievable if raising all elements to <code>nums[j]</code> costs at most <code>k</code> .	$O(n \log n)$	$O(1)$	sort so the window max is the cheapest target

Frequency Map Trick (used by 567 and 76)

Both use a `need[]` array and a `required` counter to avoid scanning the map each step.

```
// Expanding j - add s.charAt(j):
if (need[s.charAt(j)]-- > 0) required--; // was needed -> satisfied one more

// Shrinking i - remove s.charAt(i):
if (need[s.charAt(i)]++ >= 0) required++; // was 0 (used up) -> now short one
i++;
```

`need[c]` is positive when still needed, zero or negative when excess.
`required` reaches 0 exactly when all characters of `t` are covered.

Fixed Window — Sum

#643 Maximum Average Subarray I

Description: Find the contiguous subarray of length `k` with the maximum average value.
Intuition: Max average of fixed length = max sum; slide a width-`k` window and track the best running sum.

```
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        int i = 0, sum = 0, maxSum = Integer.MIN_VALUE;
        for (int j = 0; j < nums.length; j++) {
            sum += nums[j];
            if (j - i + 1 == k) {
                maxSum = Math.max(maxSum, sum);
                sum -= nums[i++];
            }
        }
        return (double) maxSum / k;
    }
}
```

Time $O(n)$ | Space $O(1)$

Fixed Window — Frequency Map

#567 Permutation in String

Description: Return true if any permutation of `s1` appears as a substring of `s2`.
Key: fixed window of size `s1.length()`. Use `required` counter — no need to compare maps.
Intuition: A permutation is just a fixed-length window whose letter counts match `s1`, so slide and check the counts.

```

class Solution {
    public boolean checkInclusion(String s1, String s2) {
        int[] need = new int[26];
        for (char c : s1.toCharArray()) need[c - 'a']++;
        int required = s1.length(), i = 0;
        for (int j = 0; j < s2.length(); j++) {
            if (need[s2.charAt(j) - 'a']-- > 0) required--;    // expand
            if (j - i + 1 == s1.length()) {
                if (required == 0) return true;
                if (need[s2.charAt(i) - 'a']++ >= 0) required++;    // shrink
                i++;
            }
        }
        return false;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Fixed Window — HashSet

#219 Contains Duplicate II

Description: Return true if any two equal values are at most k indices apart.

Key: fixed window of size k, maintained as a HashSet.

Intuition: Keep only the last k values in a set; a failed insert means a duplicate within k indices.

```

class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        Set<Integer> window = new HashSet<>();
        int i = 0;
        for (int j = 0; j < nums.length; j++) {
            if (!window.add(nums[j])) return true;    // duplicate found in window
            if (j - i + 1 > k) window.remove(nums[i++]);
        }
        return false;
    }
}

```

Time $O(n)$ | **Space** $O(k)$

Fixed Window — Monotone Deque

#239 Sliding Window Maximum

Description: Return the maximum of every contiguous subarray of size k.

Key: monotone deque stores indices in decreasing value order. Front is always the max of current window.

Intuition: A smaller value with a larger one still in the window can never be the max again, so discard it.

```

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int[] result = new int[nums.length - k + 1];
        Deque<Integer> queue = new ArrayDeque<>(); // indices, values decreasing front-back
        int i = 0;
        for (int j = 0; j < nums.length; j++) {
            while (!queue.isEmpty() && queue.peekFirst() < i) queue.pollFirst(); // evict out-of-window
            while (!queue.isEmpty() && nums[queue.peekLast()] <= nums[j]) queue.pollLast(); // evict smaller
            queue.offerLast(j);
            if (j - i + 1 == k) {
                result[i] = nums[queue.peekFirst()];
                i++;
            }
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(k)$

Max Variable Window — Sum

#1004 Max Consecutive Ones III

Description: Flip at most k zeros. Return the length of the longest subarray of 1s.

State: count of zeros in window. Invalid when `zeros > k`.

Intuition: "Flip at most k zeros" = longest window containing at most k zeros; grow, and shrink only when zeros exceed k .

```

class Solution {
    public int longestOnes(int[] nums, int k) {
        int i = 0, zeros = 0, result = 0;
        for (int j = 0; j < nums.length; j++) {
            if (nums[j] == 0) zeros++;
            while (zeros > k) {
                if (nums[i++] == 0) zeros--;
            }
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Max Variable Window — Frequency Map

#3 Longest Substring Without Repeating Characters

Description: Find the length of the longest substring with all unique characters.

State: `freq[]` array. Invalid when `freq[nums[j]] > 1` after expanding.

Intuition: A repeat appears the moment you add it, so shrink from the left until that one character is unique again.

```

class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] freq = new int[128];
        int i = 0, result = 0;
        for (int j = 0; j < s.length(); j++) {
            freq[s.charAt(j)]++;
            while (freq[s.charAt(j)] > 1) freq[s.charAt(i++)]--;
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#424 Longest Repeating Character Replacement

Description: Replace at most k characters. Find the longest substring with one repeated char.

State: `freq[] + maxFreq`. Invalid when `(window size) - maxFreq > k` (too many to replace).

Intuition: A window is valid if the non-dominant chars (`size - maxFreq`) fit within k replacements; otherwise shrink.

Note: `maxFreq` never decreases when shrinking — we only care about windows larger than current best, so a stale `maxFreq` is safe and avoids a rescan.

```

class Solution {
    public int characterReplacement(String s, int k) {
        int[] freq = new int[26];
        int i = 0, maxFreq = 0, result = 0;
        for (int j = 0; j < s.length(); j++) {
            freq[s.charAt(j) - 'A']++;
            maxFreq = Math.max(maxFreq, freq[s.charAt(j) - 'A']);
            while (j - i + 1 - maxFreq > k) freq[s.charAt(i++) - 'A']--;
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Min Variable Window — Sum

#209 Minimum Size Subarray Sum

Description: Find the minimum length subarray with $\text{sum} \geq \text{target}$.

State: running `sum`. Valid when `sum >= target` — record then shrink.

Intuition: Once the window reaches target, every extra left-trim that stays $\geq \text{target}$ gives a shorter candidate.

```

class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int i = 0, sum = 0, result = Integer.MAX_VALUE;
        for (int j = 0; j < nums.length; j++) {
            sum += nums[j];
            while (sum >= target) {
                result = Math.min(result, j - i + 1);
                sum -= nums[i++];
            }
        }
        return result == Integer.MAX_VALUE ? 0 : result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Min Variable Window — Frequency Map

#76 Minimum Window Substring

Description: Find the shortest substring of s that contains all characters of t.
State: need[] freq array + required counter. Valid when required == 0 .
Intuition: Expand until all of t is covered, then shrink as far as you can while still covering it to find the tightest window.

```
class Solution {
    public String minWindow(String s, String t) {
        int[] need = new int[128];
        for (char c : t.toCharArray()) need[c]++;
        int required = t.length(), i = 0, start = 0, minLen = Integer.MAX_VALUE;
        for (int j = 0; j < s.length(); j++) {
            if (need[s.charAt(j)]-- > 0) required--; // expand
            while (required == 0) {
                if (j - i + 1 < minLen) { minLen = j - i + 1; start = i; }
                if (need[s.charAt(i)]++ >= 0) required++; // shrink
                i++;
            }
        }
        return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);
    }
}
```

Time O(n) | Space O(1)

Side-by-Side: 567 vs 76

Both use need[] + required . One difference:

	567 (fixed, boolean)	76 (variable min, string)
Window size:	fixed = s1.length()	variable
Shrink trigger:	always when full	while required == 0
Record:	if required == 0 → true	minLen = min(minLen, j-i+1)
Return:	boolean	s.substring(start, start+minLen)

Side-by-Side: 209 vs 76

Both are Min window. One uses sum, one uses map:

	209 (sum)	76 (map)
State:	int sum	int[] need + int required
Valid condition:	sum >= target	required == 0
Expand:	sum += nums[j]	need[c]-- > 0 → required--
Shrink:	sum -= nums[i]	need[c]++ >= 0 → required++

Choosing the Template

Question asks for	Template	i moves
Fixed-size window operation	Fixed	when j-i+1 == k
Longest / maximum window	Max variable	while invalid (while)
Shortest / minimum window	Min variable	while valid (while)

Window content	State to maintain
Sum of values	int sum
Unique characters / no duplicates	int[] freq (128 or 26) or Set
Character frequency match	int[] need + int required counter
Running maximum	Deque<Integer> monotone decreasing indices

Fixed Window — Freq Array Compare

#438 Find All Anagrams in a String

Description: Given strings `s` and `p`, return a list of all start indices of `p`'s anagrams in `s`. (An anagram uses same characters with same frequencies.)

Intuition: An anagram is a fixed-length window whose letter counts equal `p`'s, so slide width- `p.length()` and compare counts.

Algorithm: Fixed-size window of length `p.length()`. Maintain a frequency count for the window. Compare with `p`'s frequency count at each step using `Arrays.equals`.

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        int[] need = new int[26], window = new int[26];
        for (char c : p.toCharArray()) need[c - 'a']++;
        int i = 0, j = 0;
        while (j < s.length()) {
            window[s.charAt(j++) - 'a']++;           // expand right
            if (j - i == p.length()) {               // ← VARIATION: fixed window size
                if (Arrays.equals(window, need)) result.add(i);
                window[s.charAt(i++) - 'a']--;       // shrink left
            }
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

Max Variable Window — K-Distinct Frequency Map

#159 Longest Substring with At Most 2 Distinct Characters

Description: Return the length of the longest substring with at most 2 distinct characters.

Intuition: Grow the window freely; whenever a third distinct character appears, drop from the left until only 2 remain.

Algorithm: Max-variable sliding window. Expand `j`; shrink `i` when the frequency map has more than 2 distinct characters.

```
class Solution {
    public int lengthOfLongestSubstringTwoDistinct(String s) {
        Map<Character, Integer> freq = new HashMap<>();
        int i = 0, max = 0;
        for (int j = 0; j < s.length(); j++) {
            freq.merge(s.charAt(j), 1, Integer::sum);
            while (freq.size() > 2) {                // ← VARIATION: shrink when >2 distinct
                char c = s.charAt(i++);
                freq.merge(c, -1, Integer::sum);
                if (freq.get(c) == 0) freq.remove(c);
            }
            max = Math.max(max, j - i + 1);
        }
        return max;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#340 Longest Substring with At Most K Distinct Characters

Description: Return the length of the longest substring with at most `k` distinct characters.

Intuition: Same as #159 but the distinct cap is `k`; shrink whenever the map holds more than `k` distinct characters.

Variation: Parameterized generalization of #159 — replace the hard-coded `2` with `k`.

```
class Solution {
    public int lengthOfLongestSubstringKDistinct(String s, int k) {
        Map<Character, Integer> freq = new HashMap<>();
        int i = 0, max = 0;
        for (int j = 0; j < s.length(); j++) {
            freq.merge(s.charAt(j), 1, Integer::sum);
            while (freq.size() > k) {                // ← VARIATION: parameterized k distinct
                char c = s.charAt(i++);
                freq.merge(c, -1, Integer::sum);
                if (freq.get(c) == 0) freq.remove(c);
            }
            max = Math.max(max, j - i + 1);
        }
        return max;
    }
}
```

Time $O(n)$ | **Space** $O(k)$

Fixed Window — Sum vs Threshold (No Division)

#1343 Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold

Description: Return the number of contiguous subarrays of size `k` whose average is greater than or equal to `threshold`.

Intuition: "Average $\geq$ threshold" over fixed width `k` is just "sum $\geq k \cdot \text{threshold}$ "; slide a width-`k` window and count the hits.

Variation: fixed-size sliding window. To avoid floating-point division, compare `windowSum >= k * threshold`.

```
class Solution {
    public int numOfSubarrays(int[] arr, int k, int threshold) {
        int target = k * threshold;                // compare sum vs k*threshold (no division)
        int windowSum = 0, count = 0;
        int i = 0;                                // i = left edge (standard i/j window)
        for (int j = 0; j < arr.length; j++) {
            windowSum += arr[j];
            if (j - i + 1 == k) {                  // window has reached size k
                if (windowSum >= target) count++;
                windowSum -= arr[i++];             // slide: drop left, stay size k
            }
        }
        return count;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

Additional Reference Problems

#30 Substring with Concatenation of All Words

Description: Given `s` and an array `words` (all of equal length), return all start indices of substrings in `s` that are a concatenation of every word in `words` exactly once, in any order.

Intuition: Every candidate substring has fixed length `words.length * wordLen`. Slide a word-aligned window: start at each offset `0..wordLen-1` and move in steps of `wordLen`, maintaining a frequency map of words seen.

Algorithm: For each starting offset, run a sliding window over words. Track a `seen` map and a `count` of matched words; shrink from the left when a word over-fills or is unknown. Record when `count == words.length`.

```

class Solution {
    public List<Integer> findSubstring(String s, String[] words) {
        List<Integer> result = new ArrayList<>();
        int wordLen = words[0].length();
        int total = words.length;
        int windowLen = wordLen * total;
        if (s.length() < windowLen) {
            return result;
        }
        Map<String, Integer> need = new HashMap<>();
        for (String w : words) {
            need.merge(w, 1, Integer::sum);
        }
        for (int offset = 0; offset < wordLen; offset++) {    // ← VARIATION: word-aligned window, step = wordLen
            int i = offset, count = 0;
            Map<String, Integer> seen = new HashMap<>();
            for (int j = offset; j + wordLen <= s.length(); j += wordLen) {
                String word = s.substring(j, j + wordLen);    // add word at right
                if (need.containsKey(word)) {
                    seen.merge(word, 1, Integer::sum);
                    count++;
                    while (seen.get(word) > need.get(word)) { // shrink: too many of this word
                        String left = s.substring(i, i + wordLen);
                        seen.merge(left, -1, Integer::sum);
                        count--;
                        i += wordLen;
                    }
                    if (count == total) {
                        result.add(i);
                    }
                } else {
                    // unknown word: reset window past it
                    seen.clear();
                    count = 0;
                    i = j + wordLen;
                }
            }
        }
        return result;
    }
}

```

Time $O(n * \text{wordLen})$ | **Space** $O(\text{words.length} * \text{wordLen})$

#187 Repeated DNA Sequences

Description: Return all 10-letter substrings that occur more than once in a DNA string `s` (characters `A`, `C`, `G`, `T`).

Intuition: Fixed window of size 10; record each substring in a set and report it the first time a duplicate insert fails.

Algorithm: Fixed-size sliding window of length 10. Use a `seen` set to detect duplicates and a `result` set to avoid reporting the same sequence twice.

```

class Solution {
    public List<String> findRepeatedDnaSequences(String s) {
        Set<String> seen = new HashSet<>();
        Set<String> result = new HashSet<>();
        int i = 0;
        for (int j = 0; j < s.length(); j++) {
            if (j - i + 1 == 10) { // ← VARIATION: fixed window size 10
                String window = s.substring(i, j + 1);
                if (!seen.add(window)) {
                    result.add(window);
                }
                i++;
            }
        }
        return new ArrayList<>(result);
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#395 Longest Substring with At Least K Repeating Characters

Description: Return the length of the longest substring of `s` such that every character in it appears at least `k` times.

Intuition: "At least `k`" is not monotone for a single window, so fix the number of distinct characters allowed. For each target `unique` from 1 to 26, run a max-window that holds exactly `unique` distinct chars, and record when all of them meet the count `k`.

Algorithm: Outer loop over `unique` (1..26). Inner sliding window tracks `count[ch-'a']`, the number of distinct chars, and how many of them reach `k`. Shrink while distinct exceeds `unique`. Record when distinct equals `unique` and all are at count `k`.

```

class Solution {
    public int longestSubstring(String s, int k) {
        int result = 0;
        for (int unique = 1; unique <= 26; unique++) { // ← VARIATION: fix distinct count to restore monotonicity
            int[] count = new int[26];
            int i = 0, distinct = 0, atLeastK = 0;
            for (int j = 0; j < s.length(); j++) {
                if (count[s.charAt(j) - 'a'] == 0) {
                    distinct++;
                }
                count[s.charAt(j) - 'a']++;
                if (count[s.charAt(j) - 'a'] == k) {
                    atLeastK++;
                }
                while (distinct > unique) { // shrink to keep at most `unique` distinct
                    if (count[s.charAt(i) - 'a'] == k) {
                        atLeastK--;
                    }
                    count[s.charAt(i) - 'a']--;
                    if (count[s.charAt(i) - 'a'] == 0) {
                        distinct--;
                    }
                    i++;
                }
                if (distinct == unique && atLeastK == unique) {
                    result = Math.max(result, j - i + 1);
                }
            }
        }
        return result;
    }
}

```

Time $O(26 * n)$ | **Space** $O(1)$

#487 Max Consecutive Ones II

Description: Given a binary array `nums`, return the maximum number of consecutive 1s if you may flip at most one 0.

Intuition: Identical to #1004 with `k = 1`: longest window containing at most one zero. Grow, and shrink only when a second zero enters.

Algorithm: Max-variable sliding window. Track `zeros`; shrink from the left while `zeros > 1`.

```
class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {
        int i = 0, zeros = 0, result = 0;
        for (int j = 0; j < nums.length; j++) {
            if (nums[j] == 0) {
                zeros++;
            }
            while (zeros > 1) {                // ← VARIATION: at most one zero (k = 1)
                if (nums[i] == 0) {
                    zeros--;
                }
                i++;
            }
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#689 Maximum Sum of 3 Non-Overlapping Subarrays

Description: Find three non-overlapping subarrays of length `k` with maximum total sum and return their starting indices (lexicographically smallest on ties).

Intuition: Precompute every window sum of size `k`, then for each middle window pick the best left window to its left and the best right window to its right.

Algorithm: Fixed window sums into `windowSum`. Build `left[m]` = index of best window in `[0..m]` and `right[m]` = index of best window in `[m..end]`. Sweep the middle window and combine.

```

class Solution {
    public int[] maxSumOfThreeSubarrays(int[] nums, int k) {
        int n = nums.length;
        int numWindows = n - k + 1;
        int[] windowSum = new int[numWindows];
        int i = 0, sum = 0;
        for (int j = 0; j < n; j++) {
            // fixed window sums of size k
            sum += nums[j];
            if (j - i + 1 == k) {
                windowSum[i] = sum;
                sum -= nums[i];
                i++;
            }
        }
        int[] left = new int[numWindows];
        int best = 0;
        for (int m = 0; m < numWindows; m++) {
            if (windowSum[m] > windowSum[best]) {
                best = m;
            }
            left[m] = best;
        }
        int[] right = new int[numWindows];
        best = numWindows - 1;
        for (int m = numWindows - 1; m >= 0; m--) {
            if (windowSum[m] >= windowSum[best]) {
                // >= keeps smallest index on ties
                best = m;
            }
            right[m] = best;
        }
        int[] result = new int[]{-1, -1, -1};
        int maxTotal = -1;
        for (int mid = k; mid + k <= numWindows - 1; mid++) {
            // ← VARIATION: middle window scan with k-gaps
            int l = left[mid - k];
            int r = right[mid + k];
            int totalSum = windowSum[l] + windowSum[mid] + windowSum[r];
            if (totalSum > maxTotal) {
                maxTotal = totalSum;
                result = new int[]{l, mid, r};
            }
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#713 Subarray Product Less Than K

Description: Return the number of contiguous subarrays whose product of all elements is strictly less than k .

Intuition: Max-variable window: grow the window while the product stays below k ; every valid window ending at j contributes $j - i + 1$ new subarrays.

Algorithm: Maintain a running product. Shrink from the left while $\text{product} \geq k$. Add $j - i + 1$ to the count at each step.

```

class Solution {
    public int numSubarrayProductLessThanK(int[] nums, int k) {
        if (k <= 1) {
            return 0;
        }
        int i = 0, product = 1, result = 0;
        for (int j = 0; j < nums.length; j++) {
            product *= nums[j];
            while (product >= k) {                // shrink while window invalid
                product /= nums[i];
                i++;
            }
            result += j - i + 1;                  // ← VARIATION: count subarrays ending at j
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#727 Minimum Window Subsequence

Description: Return the minimum-length substring (window) of `s1` such that `s2` is a subsequence of it. On ties, return the leftmost.

Intuition: Walk forward matching `s2` as a subsequence; once fully matched at index `j`, walk backward to tighten the start, giving the smallest window ending at `j`. Restart the forward scan just past that start.

Algorithm: Two-pointer subsequence match. Forward pass advances `k` over `s2`; when `k` reaches the end, scan back to find the matching start, record the window, then resume scanning after the start position.

```

class Solution {
    public String minWindow(String s1, String s2) {
        int n = s1.length(), m = s2.length();
        int start = -1, minLen = Integer.MAX_VALUE;
        int k = 0;                                // pointer into s2
        for (int j = 0; j < n; j++) {
            if (s1.charAt(j) == s2.charAt(k)) {    // ← VARIATION: subsequence match, not contiguous
                k++;
                if (k == m) {                      // matched all of s2 ending at j
                    int end = j + 1;
                    k--;
                    int i = j;                    // walk back to the tightest start
                    while (k >= 0) {
                        if (s1.charAt(i) == s2.charAt(k)) {
                            k--;
                        }
                        i--;
                    }
                    i++;                          // i now points at window start
                    k++;
                    if (end - i < minLen) {
                        minLen = end - i;
                        start = i;
                    }
                    j = i;                        // resume scan just after the start
                    k = 0;
                }
            }
        }
        return start == -1 ? "" : s1.substring(start, start + minLen);
    }
}

```

Time $O(n * m)$ | **Space** $O(1)$

#1044 Longest Duplicate Substring

Description: Return any longest substring that appears at least twice in `s` (overlaps allowed); return `""` if none.

Intuition: Binary search the answer length: if a duplicate of length `L` exists, a duplicate of any shorter length also exists. Check each candidate length with a rolling hash over a fixed-size window.

Algorithm: Binary search on length `L`. For each `L`, slide a window of size `L`, compute its rolling (Rabin-Karp) hash, and store hashes in a set; a collision (verified by substring compare) means a duplicate of length `L` exists.

```
class Solution {
    public String longestDupSubstring(String s) {
        int n = s.length();
        long mod = (1L << 61) - 1;
        long base = 131;
        int lo = 1, hi = n - 1;
        String result = "";
        while (lo <= hi) {
            // + VARIATION: binary search the window size
            int len = lo + (hi - lo) / 2;
            int start = search(s, len, base, mod);
            if (start != -1) {
                result = s.substring(start, start + len);
                lo = len + 1;
            } else {
                hi = len - 1;
            }
        }
        return result;
    }

    private int search(String s, int len, long base, long mod) {
        long hash = 0, power = 1;
        for (int j = 0; j < len; j++) {
            // hash of first window [0, len)
            hash = (hash * base + s.charAt(j)) % mod;
        }
        for (int j = 0; j < len - 1; j++) {
            power = (power * base) % mod;
            // base^(len-1)
        }
        Map<Long, List<Integer>> seen = new HashMap<>();
        seen.computeIfAbsent(hash, x -> new ArrayList<>()).add(0);
        int i = 1;
        // window start
        for (int j = len; j < s.length(); j++) {
            // slide fixed window of size len
            hash = (hash - s.charAt(i - 1) * power % mod + mod) % mod;
            hash = (hash * base + s.charAt(j)) % mod;
            String window = s.substring(i, j + 1);
            List<Integer> candidates = seen.get(hash);
            if (candidates != null) {
                for (int idx : candidates) {
                    if (s.substring(idx, idx + len).equals(window)) {
                        return i;
                    }
                }
            }
            seen.computeIfAbsent(hash, x -> new ArrayList<>()).add(i);
            i++;
        }
        return -1;
    }
}
```

Time $O(n \log n)$ average | **Space** $O(n)$

#1838 Frequency of the Most Frequent Element

Description: Given `nums` and `k` operations (each increments one element by 1), return the maximum possible frequency of any single value.

Intuition: Sort the array; the cheapest target to raise a window of elements to is its maximum (the rightmost in a sorted window). A window `[i, j]` is achievable if raising all elements to `nums[j]` costs at most `k`.

Algorithm: Sort. Max-variable sliding window over a running `sum`. Cost to level the window up to `nums[j]` is `(long)(j - i + 1) * nums[j] - sum`; shrink while that exceeds `k`. Track the largest window size.

```
class Solution {
    public int maxFrequency(int[] nums, int k) {
        Arrays.sort(nums);
        // ← VARIATION: sort so the window max is the cheapest target
        int i = 0, result = 0;
        long sum = 0;
        for (int j = 0; j < nums.length; j++) {
            sum += nums[j];
            while ((long) (j - i + 1) * nums[j] - sum > k) { // shrink while leveling cost exceeds k
                sum -= nums[i];
                i++;
            }
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}
```

Time $O(n \log n)$ | **Space** $O(1)$